

Postal Delivery Operations System

System guide: the backend, the admin panel and the Postman field app - as they are today

How an address becomes a beat and a postman, how territories are checked, how routes are planned, and what is measured, tested and not done.

Generated from the source and the measured result files in the repository on 2026-09-21

Contents

1. What this system is	2
The life of a delivery	2
2. Roles, access and security	2
3. Data model (the tables that matter)	3
4. Address to beat matching	4
Normalization and score	4
Decision: name first, weak geocodes never override	4
5. Territories	4
6. Geocoding and address learning	5
7. Beat-list import	5
8. Route optimization	6
9. The admin panel	6
10. The Postman app	6
11. Configuration	7
12. Testing and CI	7
13. What was measured	7
14. Known limitations	9

1. What this system is

The system runs a post office's delivery round from the moment a parcel list arrives to the moment a postman completes a delivery. Three parts share one backend and one PostgreSQL / PostGIS database, which is the only source of truth:

- **Backend** - Express + TypeScript + Prisma. Owns the data and every rule: address matching, confidence thresholds, the status state machine, password rules, proof requirements, route planning, notifications.
- **Admin panel** - React + Vite for post office staff: beats and territories, imports, assignment exceptions, postmen, deliveries, the map. It displays and asks; it decides nothing the backend decides.
- **Postman app** - Expo / React Native: today's round in route order, navigation, status, proof photo, an offline queue with conflict handling, history, notifications.

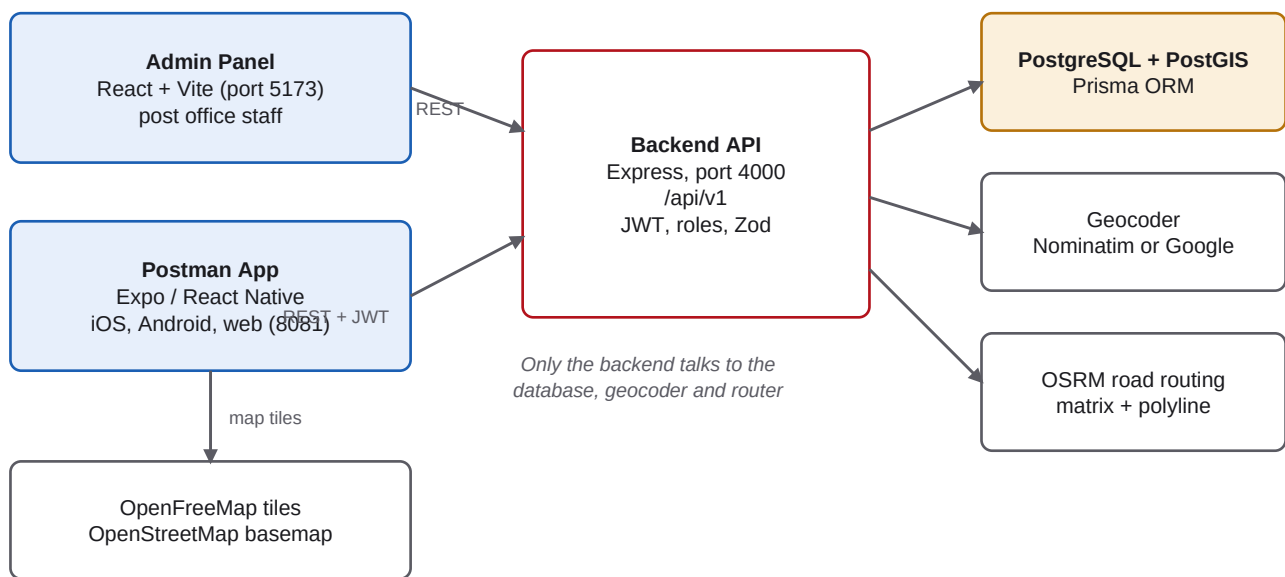


Figure 1. Both clients call the same REST API. Only the backend touches the database, the geocoder, the routing engine and Expo's push service.

The life of a delivery

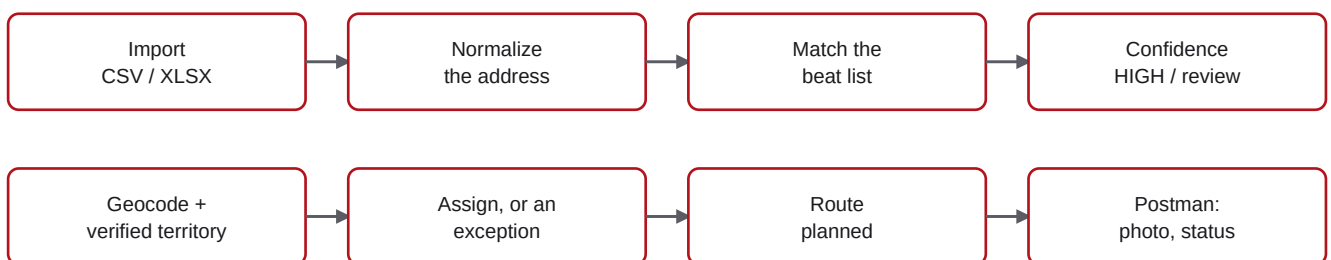


Figure 2. The order matters: the beat list comes first; a geocode supports it and decides alone only when it is precise and lies in exactly one verified territory. Anything uncertain becomes an assignment exception, never a guess.

2. Roles, access and security

Three roles: SUPER_ADMIN (every post office), ADMIN (one post office), POSTMAN (their own deliveries). Every request carries a JWT; the role and post office in it decide what the caller sees, and the API - not the panels - enforces it (resolvePostOfficeScope, assertOwnsResource). A postman's own id is always derived from the token, never from anything the client sends.

- **First sign-in.** An account created or reset by someone else has a **temporary password** (mustChangePassword). Its token carries a restriction: the server refuses every endpoint except reading who you are, changing the password and signing out (403 PASSWORD_CHANGE_REQUIRED). Both apps show a *Choose a new password* screen instead of the application. Changing it ends every other session. Rules: 10+ characters, a letter and a number, not the email name, not a well-known password. No password is written in the source; the seed prints a random one once.
- **Sign-in is rate-limited** (20 attempts per 15 minutes per address by default; AUTH_RATE_LIMIT). Passwords are hashed with argon2id; refresh tokens are stored hashed and rotated.
- **Proof photos are private.** They live in a folder that is never served as static files, under random names; the only way to read one is an authenticated request from the delivery's own postman or the post office's administrators (Cache-Control: private, no-store). The bytes are checked to be a real JPEG / PNG / WebP, whatever the file is called.
- **Audit log.** Logins, imports, verification, territory changes, assignment overrides, status changes, proof added, password changed, proof setting changed - who, what, when.

3. Data model (the tables that matter)

Table	Holds
PostOffice	office, location, proofMode (NONE PHOTO)
Beat	number, name, boundary geometry(Polygon, 4326), verificationStatus (VERIFIED / PENDING_VERIFICATION / NEEDS_REVIEW), who verified
BeatLocality	the beat directory: one row per (beat, locality, main area, pincode) with normalised forms - what an address is matched against
Address	the address, coordinates, geocodingPrecision (HOUSE / STREET / AREA / PINCODE / NONE), provider metadata, normalised key
Delivery	status, beat, postman, weightKg, priority, assignmentMethod / assignmentConfidence / assignmentEvidence
AssignmentException	reason, message, suggestedBeatId, confidence, location quality, evidence, resolution
AddressLocation / AddressLocationSample	address learning: the median point per learnable address, and every GPS sample with its accept / reject reason
DeliveryProof	the proof photo's metadata and private storage key (never the bytes, never a URL)
PushToken, Notification	phones registered for push; every message a user was sent (title, read time)
Postman, PostmanBeatAssignment, Route, RouteStop, DeliveryAssignmentHistory, DeliveryStatusHistory, AuditLog, User, ...	the operational records

All schema changes are additive migrations in backend/prisma/migrations (no reset, no destructive change); CI checks that the migrations and the Prisma schema agree.

4. Address to beat matching

The beat list comes first because it needs no coordinates and is what the post office actually knows: which localities each beat covers. Full detail and the validation are in backend/ADDRESS_MATCHING.md.

Normalization and score

- **Normalization:** case, punctuation, abbreviations (RD→ROAD, W→WEST, BLDG→BUILDING), spelling variants (NIVAS/NIWAS), dotted initials (L.B.S.→LBS), pincode. "Farid Nagar, Bhandup West, Mumbai" and "Farid Nagar Bhandup W Mumbai" become the same tokens.
- **Score** = named evidence, capped at 100: locality up to 60 (a whole segment 100 %, fuzzy 92 %, phrase 93 %, all words in one segment 75 %, partial 55 %), the delivery's own locality field +15, a main area up to 30, the post office's area +5, the pincode +5. Common words (ROAD, NAGAR) count for less; short words never match fuzzily; there is **no negative evidence** (the directory is incomplete).
- **Levels:** HIGH (score ≥ 65 and 15 points clear of the next beat), AMBIGUOUS (another beat within 15), MEDIUM (45-64), LOW, NONE. The thresholds were swept on the ground truth, not picked: **all 85 combinations swept gave zero wrong automatic assignments.**

Decision: name first, weak geocodes never override

Name evidence	Location evidence	Result
HIGH	none / area / pincode / street	Assign - method NAME
HIGH	house-level, inside the same verified territory	Assign - NAME_AND_TERRITORY
HIGH	house-level, inside a <i>different</i> verified territory	Exception AMBIGUOUS_MATCH (a conflict for a person, not an override)
MEDIUM	usable location inside that beat's verified territory	Assign - NAME_AND_TERRITORY
MEDIUM / AMBIGUOUS	otherwise	Exception with the suggestion and the contenders
LOW / NONE	house-level, inside exactly one verified territory	Assign - TERRITORY (confidence 80, not validated)
LOW / NONE	house-level, inside several / none	Exception MULTIPLE_BEAT_MATCH / NO_BEAT_MATCH - never a random pick
LOW / NONE	area or pincode level	Exception WEAK_LOCATION: "Location could not be determined precisely enough to assign this delivery automatically."

- **Every automatic assignment is explained:** the method, the confidence and the evidence (which locality, which main area, how it matched, the location quality, the territories) are stored with the delivery and shown on its page.
- **Assignment Exceptions page:** delivery, recipient, address (each part once), suggested beat (*B20 - Farid Nagar*), confidence (*92% - High*), reason, location quality, and the actions **Assign** (accept the suggestion), **Choose Beat**, **Ignore**. A manual decision is recorded as MANUAL and is never overwritten by a re-run; a beat nobody covers keeps a *no postman* exception open.

5. Territories

A beat's territory is a PostGIS polygon. The panel shows four indicators: ✓ **Verified**, ■ **Needs verification**, ■ **Missing territory** and ■ **Overlapping B21** (a heavy dashed outline on the map). Only **verified** territories take part in matching; a pending one is a draft.

Checked when a territory is saved	Rule
Valid shape	PostGIS ST_IsValid - a self-intersecting outline is refused with PostGIS's reason

Checked when a territory is saved	Rule
Coordinates	WGS84 longitude / latitude in range
Size	1 000 m ² to 25 km ²
Position	centre within 30 km of its post office (catches a swapped latitude / longitude)
Overlap	with any other active beat by more than 25 m ² : refused with " <i>Beat 30 overlaps Beat 20 (1,234 m²)</i> " unless the administrator confirms it is intended (also required to verify an overlapping beat)

An address inside two verified territories is an exception, never a random pick. GET `/beats/territory-report` gives the office's counts (verified / needs verification / missing / overlapping) and every overlapping pair. Nothing infers "verified": the Bhandup West territories inferred from OpenStreetMap anchors are imported *pending*.

6. Geocoding and address learning

GeocodingService is provider-independent (Nominatim, Google, or none). A result carries the coordinates, the provider's confidence, the components it recognised and, above all, the **precision**: HOUSE (a building), STREET, AREA (the centre of a suburb), PINCODE (the centre of a postal area) or NONE. The precision comes from what the provider says the match *is*, capped by the query tier (a house-like match returned for a fallback "area, city" query is AREA).

- Only HOUSE is strong evidence; AREA and PINCODE never assign anything. A pincode-level point is not routed (a postman cannot drive to the middle of a postal area): it is left out and reported.
- Address learning.** A delivery completed with the postman's own fresh GPS fix teaches the system where that address is; the next delivery to it reuses the learned point before any geocoder is asked. A fix is accepted only with a stated accuracy of 50 m or better, within 1 km of an existing precise geocode, and within 150 m of what was learned so far; the stored point is the median; one sample is STREET quality, two that agree are HOUSE quality. Only an address with a house / unit number and at least two other words is learnable. Rejected samples are kept with the reason.

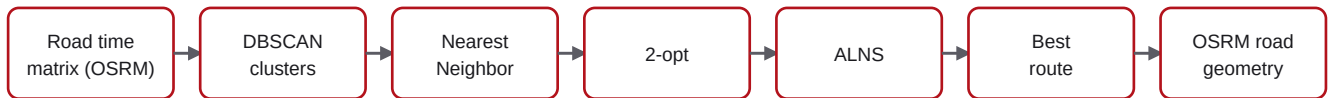
7. Beat-list import

Workflow in the admin panel: **upload** → **validate** → **preview** → **map columns** → **review** → **import** → **verify**. The real Bhandup West list is a PDF, which is not parsed; the importer takes CSV / XLSX and does not assume a format (columns are recognised by common names and can be remapped). Two shapes: one row per beat, or one row per (beat, locality) where the rows of one beat number are ONE beat with several localities, stored as BeatLocality records (not opaque JSON).

- The preview counts, before anything is written: rows, new beats, existing beats receiving localities, locality records, repeated rows, unknown post offices, missing beat numbers, invalid rows, beats with no locality or territory.
- No row is silently discarded:** a repeated (beat, locality, main area) is listed as *Skipped* with the row it repeats; every problem row is in the downloadable report. After the import, deliveries waiting for a beat are matched against the new directory.

8. Route optimization

There is **one** production pipeline and no algorithm selector anywhere (API, panel or app):



- **Cost** = driving time + load penalty + priority penalty, with arrival times including service time. **Load is a real weight** (`Delivery.weightKg`): a parcel count is never treated as a weight; with no weights the load term is switched off and the route says so (`loadBasis: NONE`). Priority (HIGH 1, URGENT 3) is in the cost, the Nearest Neighbor key and the ALNS insertion delta - tested: an URGENT stop 315 s away goes before a NORMAL one 110 s away.
- **ALNS** (Ropke-Pisinger): destroy operators random / worst / related / cluster, repair operators greedy / regret-2 / regret-3, adaptive weights, simulated-annealing acceptance, the best route tracked separately (never dearer than the 2-opt route), exact $O(1)$ insertion deltas from prefix / suffix sums.
- **Clusters stay together** (measured): with plain deliveries the drive time is the same with clusters kept or free (-0.3 % to +0.2 %) and 1.7-4x faster; with priorities and weights live, freeing the clusters gives a lower cost but a 9-15 % *longer* drive. Route quality is not hurt, so the default stays.
- **Time windows are not implemented**: they make lateness non-linear in the arrival time, which breaks the closed-form insertion cost that makes ALNS fast, and no data carries a window. Planning is **synchronous** (about 50 ms for 25 stops, 200 ms for 50, 800 ms for 100 and 850 ms for 200; ALNS capped at 1.5 s): a background job would add machinery for no visible gain at beat sizes.

9. The admin panel

- **Map** (Operations Map): beats with their territory state, search, draw / edit a territory, verify, upload a beat list; territory checks and the overlap confirmation are the server's.
- **Assignment Exceptions** as described above; **Deliveries** and the delivery page (status, history, how it was assigned with the evidence, the proof photo, fetched with the administrator's token); **Imports** and **Data quality**; **Postmen**; **Reports**; **Audit log**.
- **Dashboard**: counts, and the *Proof of delivery* setting of the post office (nothing extra, or a photo). With a photo required the server refuses DELIVERED without it - for the postman and for an administrator alike.
- **Forced first-login password change** as in section 2.

10. The Postman app

- **Tabs**: Home, Deliveries (Today | History), Map, Account. Addresses are shown once (*21 Farid Nagar / Bhandup West / Mumbai*); the tab-bar height follows the device's own bottom inset so the labels are never clipped.
- **Status** only through the server's state machine: an ASSIGNED parcel offers *Start Delivery*, never *Mark Delivered*.
- **Proof of delivery**. When the post office requires a photo, *Mark Delivered* opens the camera first (camera only), keeps the photo inside the app, and sends it before the status change; offline, the photo and the change are queued together and replayed photo-first. The postman's fresh GPS fix goes with the completed delivery for address learning. Not implemented: OTP (needs an SMS provider) and signature.
- **Offline**. One persistent queue for every change, replayed oldest first; a network failure stops the replay and keeps the rest; a rejected change is never forced (the app re-reads the server and shows a conflict with the reason). **History works offline** (the last first page per period is saved and marked as such) and its top lists every change not on the server yet: *Waiting to sync* or *Conflict*.

- **Notifications.** The backend stores a message and pushes it through Expo for: new assignments (batched when an import assigns many), an urgent delivery, a reassignment, a route change, a message from the office. The Notifications screen is the source of truth. Push to a device needs a real phone and a build with the project's push credentials, so it is **not verified here**; registration, sending and the handling of a dead token are tested against a fake of Expo's API.

11. Configuration

Variable	Default	Meaning
DATABASE_URL, JWT_ACCESS_SECRET, JWT_REFRESH_SECRET	-	required
GEOCODING_PROVIDER	nominatim	nominatim google (GOOGLE_GEOCODING_API_KEY) none
OSRM_BASE_URL	empty	road routing; empty = flagged straight-line estimates; the public demo server is for development
ROUTE_ALNS_PRESERVE_CLUSTERS, ROUTE_ALNS_MAX_MS, ...	true, 1500	tune (never replace) the pipeline
PUSH_ENABLED, EXPO_ACCESS_TOKEN, PUSH_BATCH_MS	true, empty, 20000	push through Expo; the token is a secret and optional; batching of assignment notifications
MAX_PROOF_PHOTO_MB	5	proof photo size limit
AUTH_RATE_LIMIT	20	sign-in attempts per 15 minutes per address
SEED_PASSWORD	random	the seed's account password; printed once when not set

12. Testing and CI

Every number below is a result of running the suite on this code; CI ([.github/workflows/ci.yml](#)) runs the same commands on every push and pull request, with nothing skipped.

Suite	Result	What it covers
Backend unit (vitest)	407 tests, 24 files	normalization, matcher, decision, ground-truth validation, learning, precision, import (structured lists, weights), routing / ALNS / clusters / priority / load, status machine, push, mocked-storage API tests
Backend integration (real PostgreSQL + PostGIS)	41 tests, 4 files	the whole assignment pipeline, territory validation and overlaps on PostGIS, proof (private storage, access control, enforcement), forced password change
Admin panel (vitest)	9 tests	territory indicators, exceptions page, forced password change, proof setting
Postman app (jest)	235 tests, 30 suites	state machine, offline queue, sync and conflicts (incl. photo-first), history offline, address display, proof flow
Browser tests (Chrome, real backend)	45 checks	admin sign-in, forced password change, beat upload, beat verification, delivery assignment and exceptions, postman sign-in, route order, delivery completion

Also: TypeScript strict checks on all three projects, ESLint on all three (0 errors), the Prisma migration/schema drift check, and expo config. Regression coverage of the areas listed for the release: authentication, PostGIS, beat and delivery assignment, route optimization, the offline queue, conflict handling, status transitions and audit logging.

13. What was measured

The same 130 Bhandup West deliveries as before (Accuracy-Test-Report.pdf has the detail and the charts).

	Previous system	Now (all 130)	Fair test (own row held out)
Correct / wrong / sent to an administrator	0 / 5 / 125	129 / 0 / 1	91 / 0 / 39

	Previous system	Now (all 130)	Fair test (own row held out)
Accuracy of all 130	0 %	99.2 %	70.0 %

- Methods used on the 130: NAME 129, NAME_AND_TERRITORY 0, TERRITORY 0, MANUAL 0, exceptions 1. The geocoder returned only area- or pincode-level points, so the geocode / territory methods never fired.
- **The full-directory figure is partly circular** (the test file's directory lists each delivery's own main area); the hold-out figure is what the matcher itself achieves. The coordinates are OpenStreetMap anchors, not house positions: this is not a measurement of house-level accuracy.
- Routing on real road times, 130 stops on 17 distinct points: input order 372.4 km, Nearest Neighbor 120, NN + 2-opt 115.64, DBSCAN + NN + 2-opt 115.62, + **ALNS 96.1 km** (14.74 % cheaper than 2-opt). On five small rounds (4-5 distinct places) ALNS found nothing better than 2-opt - reported as it is.

14. Known limitations

- **The real beat list is a PDF** and is not parsed; the importer takes CSV / XLSX. The beat directory used here is the supplied CSV (201 rows).
- **The accuracy test's ground truth is the file's own layout** and its directory includes each delivery's own locality: use the hold-out figure. There is no house-level ground truth, so the TERRITORY confidence (80), the geocode-precision rules and address learning are covered by tests, not validated on real positions. The MEDIUM confidence band was never reached by the test data.
- **Time windows** are not implemented (no data; they break the ALNS insertion closed form). **OTP and signature proof** are not implemented (no SMS provider; no requirement).
- **Push delivery to a phone** cannot be verified without a device and Expo's servers. The native map screen has not been run on a device in this environment; the web build of the app was exercised end to end in a browser.
- **OSRM and Nominatim** are external services (the public demo servers are development-only); without OSRM routes are flagged straight-line estimates.
- **Existing accounts** created before the forced-change feature keep their current passwords until an administrator resets them or sets `mustChangePassword` for them.
- **CI** runs on GitHub; its result is reported by GitHub after the push, not by this document.

This guide describes the system as it is today. Where something is not done (time windows, OTP, PDF parsing, push verification on a device), it says so.